Lab - 1 | 22 January 2026

↳ 6 cells hidden

Lab - 2 | 29 January 2026

↳ 14 cells hidden

Lab - 3 | 05 February 2026

↳ 4 cells hidden

Lab 4 | 12 February 2026

Lab 5 | 19 February 2026

↳ 2 cells hidden

Lab - 6 | 26 February 2026

↳ 7 cells hidden

Lab - 7 | 05 March 2026

House Price Prediction (Regresssion Model)

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tra
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
```

```
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
```

```
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lati
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     3
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     3
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     3
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     3
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     3

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential"

Layer (type)	Output Shape	P
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 64)	
dense_2 (Dense)	(None, 32)	
dense_3 (Dense)	(None, 1)	

```
 Total params: 11,521 (45.00 KB)
 Trainable params: 11,521 (45.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 1.9615 - mae: 0.765
Epoch 2/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3759 - mae: 0.429
Epoch 3/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3416 - mae: 0.406
Epoch 4/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3284 - mae: 0.398
Epoch 5/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3043 - mae: 0.385
Epoch 6/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2990 - mae: 0.380
Epoch 7/100
```

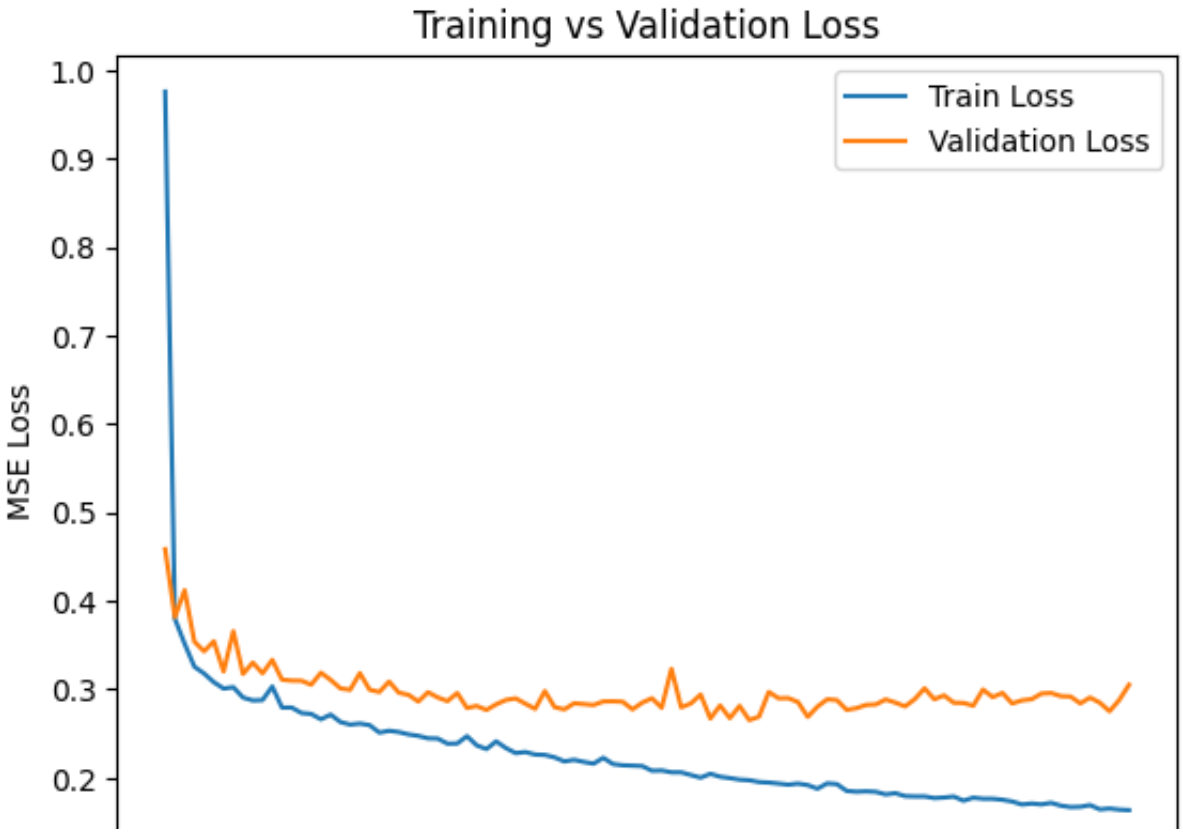
```
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2966 - mae: 0.371
Epoch 8/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2946 - mae: 0.378
Epoch 9/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2915 - mae: 0.372
Epoch 10/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2862 - mae: 0.367
Epoch 11/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2885 - mae: 0.369
Epoch 12/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2972 - mae: 0.367
Epoch 13/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2804 - mae: 0.368
Epoch 14/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2823 - mae: 0.361
Epoch 15/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2702 - mae: 0.356
Epoch 16/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2718 - mae: 0.358
Epoch 17/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2677 - mae: 0.354
Epoch 18/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2622 - mae: 0.349
Epoch 19/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2531 - mae: 0.347
Epoch 20/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.2622 - mae: 0.351
Epoch 21/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.2523 - mae: 0.344
Epoch 22/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2586 - mae: 0.344
Epoch 23/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2515 - mae: 0.343
Epoch 24/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2411 - mae: 0.335
Epoch 25/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2456 - mae: 0.339
Epoch 26/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2517 - mae: 0.343
Epoch 27/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2513 - mae: 0.339
Epoch 28/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2421 - mae: 0.336
Epoch 29/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2428 - mae: 0.340
Epoch 30/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2372 - mae: 0.334
Epoch 31/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2293 - mae: 0.328
Epoch 32/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2398 - mae: 0.334
Epoch 33/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2338 - mae: 0.329
Epoch 34/100
```

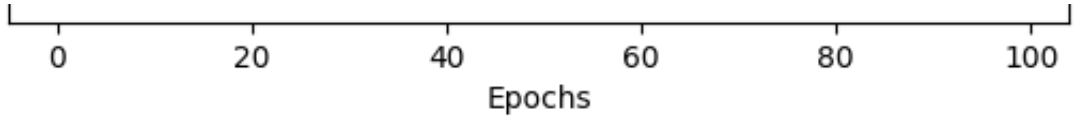
```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2319 - mae: 0.327
Epoch 35/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2298 - mae: 0.324
Epoch 36/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2382 - mae: 0.331
Epoch 37/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2253 - mae: 0.326
Epoch 38/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2275 - mae: 0.326
Epoch 39/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2293 - mae: 0.323
Epoch 40/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2216 - mae: 0.323
Epoch 41/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2149 - mae: 0.317
Epoch 42/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2143 - mae: 0.313
Epoch 43/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2152 - mae: 0.320
Epoch 44/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2213 - mae: 0.324
Epoch 45/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2145 - mae: 0.319
Epoch 46/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2211 - mae: 0.320
Epoch 47/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2070 - mae: 0.309
Epoch 48/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2085 - mae: 0.315
Epoch 49/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2167 - mae: 0.316
Epoch 50/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2167 - mae: 0.319
Epoch 51/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2061 - mae: 0.311
Epoch 52/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2011 - mae: 0.310
Epoch 53/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2037 - mae: 0.309
Epoch 54/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2085 - mae: 0.313
Epoch 55/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2062 - mae: 0.309
Epoch 56/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1934 - mae: 0.301
Epoch 57/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2027 - mae: 0.306
Epoch 58/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1945 - mae: 0.303
Epoch 59/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2010 - mae: 0.307
Epoch 60/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2021 - mae: 0.308
Epoch 61/100
```

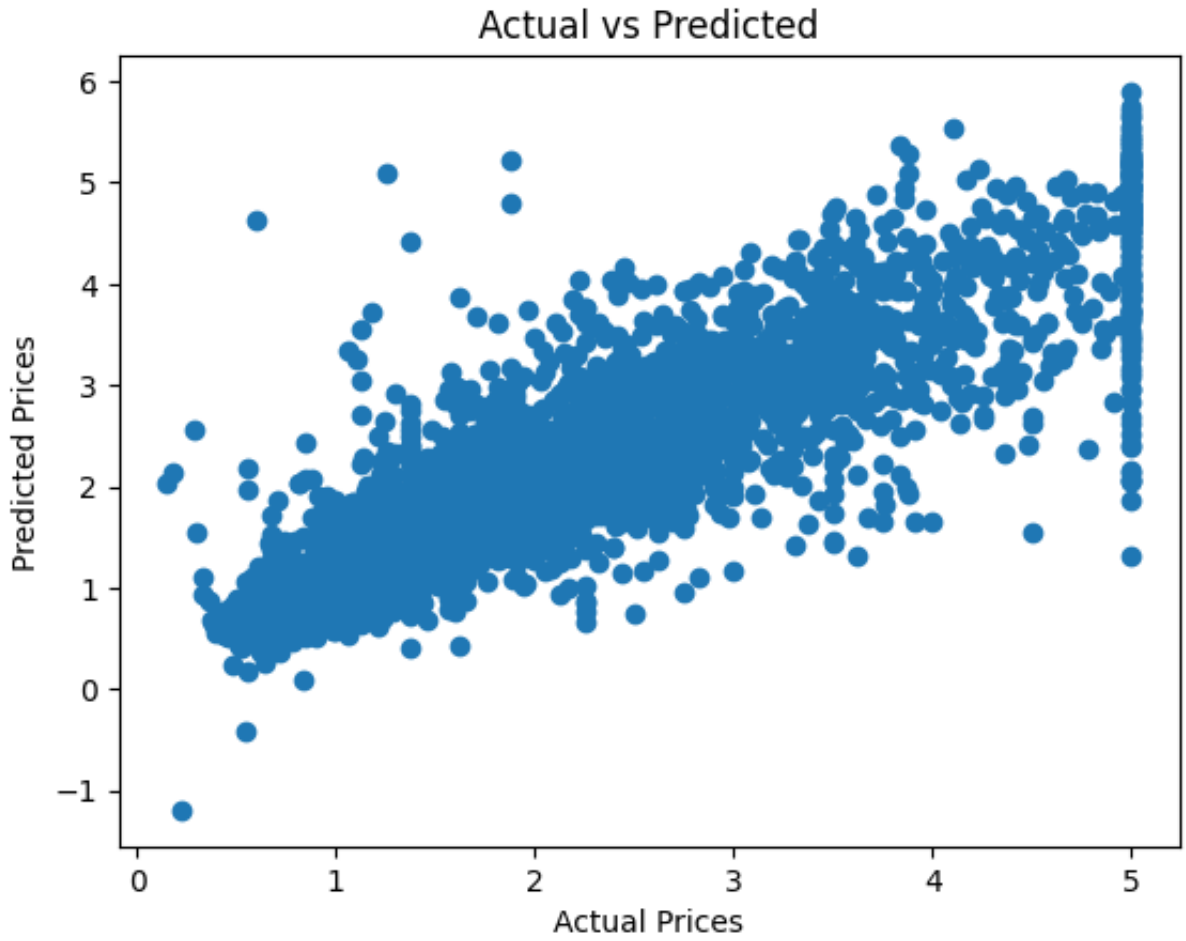
```
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1975 - mae: 0.303
Epoch 62/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1926 - mae: 0.301
Epoch 63/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1928 - mae: 0.303
Epoch 64/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1888 - mae: 0.299
Epoch 65/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2016 - mae: 0.306
Epoch 66/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1935 - mae: 0.302
Epoch 67/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1889 - mae: 0.296
Epoch 68/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1799 - mae: 0.292
Epoch 69/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1870 - mae: 0.296
Epoch 70/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1922 - mae: 0.300
Epoch 71/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1856 - mae: 0.296
Epoch 72/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1838 - mae: 0.295
Epoch 73/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1849 - mae: 0.296
Epoch 74/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1793 - mae: 0.292
Epoch 75/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.1751 - mae: 0.289
Epoch 76/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1783 - mae: 0.289
Epoch 77/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1811 - mae: 0.291
Epoch 78/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1754 - mae: 0.287
Epoch 79/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1807 - mae: 0.291
Epoch 80/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1695 - mae: 0.285
Epoch 81/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1846 - mae: 0.295
Epoch 82/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1738 - mae: 0.289
Epoch 83/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1761 - mae: 0.287
Epoch 84/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1707 - mae: 0.286
Epoch 85/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1756 - mae: 0.288
Epoch 86/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1765 - mae: 0.289
Epoch 87/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1831 - mae: 0.292
Epoch 88/100
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1762 - mae: 0.290
Epoch 89/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.1661 - mae: 0.282
Epoch 90/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1724 - mae: 0.287
Epoch 91/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.1647 - mae: 0.284
Epoch 92/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1673 - mae: 0.283
Epoch 93/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1635 - mae: 0.281
Epoch 94/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1663 - mae: 0.284
Epoch 95/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1720 - mae: 0.287
Epoch 96/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1737 - mae: 0.286
Epoch 97/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1521 - mae: 0.273
Epoch 98/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - loss: 0.1589 - mae: 0.278
Epoch 99/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1600 - mae: 0.277
Epoch 100/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1657 - mae: 0.285
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.2849 - mae: 0.353
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

Test MAE: 0.3528080880641937
RMSE: 0.5334247070152569
R2 Score: 0.7828602193825904
```







Printing after Normalization 90/10 Split

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
```

```
# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert back to DataFrame for readable printing
X_train_scaled_df = pd.DataFrame(X_train, columns=df.drop("Price",
X_test_scaled_df = pd.DataFrame(X_test, columns=df.drop("Price", ax

print("\nAfter Normalization (First 5 rows of Training Data):")
print(X_train_scaled_df.head())

print("\nAfter Normalization (First 5 rows of Test Data):")
print(X_test_scaled_df.head())

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tr
```

```
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.005),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
```

```
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lati
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     3
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     3
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     3
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     3
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     3

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422

After Normalization (First 5 rows of Training Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -0.018837  0.983513 -0.066315   0.072017   -0.009699 -0.006443 -0.
1  2.185237  1.856793  0.712744  -0.137047   -0.132519 -0.044325  0.
2 -0.106583  0.110233 -0.705284  -0.025634    1.097431 -0.068481  0.
3 -0.377698 -1.001215  0.075494  -0.127940   -0.557123 -0.028728  1.
4  0.558387 -1.080604  0.358362  -0.223439   -0.014086  0.010648  0.

   Longitude
0   0.641303
1  -1.439758
2  -1.285051
3  -0.815939
4  -0.087318

After Normalization (First 5 rows of Test Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -1.158483 -0.286713 -0.493531  -0.154095   -0.030754  0.072677  0.
1 -0.709126  0.110233 -0.156462   0.199021    0.121016 -0.036911 -0.
2 -0.207597  1.856793 -0.579091   0.183314   -0.102691 -0.157646  1.
3  0.985701 -0.921826  0.290845  -0.158389    0.243835  0.033057 -0.
4 -0.078145  0.427789  0.024015  -0.142229   -0.319380 -0.054860  0.

   Longitude
0   0.281983
1   0.057408
```

```
1   0.057408
2  -1.429777
3   0.426709
4  -1.175258
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential_4"

Layer (type)	Output Shape	P
dense_16 (Dense)	(None, 128)	
dense_17 (Dense)	(None, 64)	
dense_18 (Dense)	(None, 32)	
dense_19 (Dense)	(None, 1)	

```
 Total params: 11,521 (45.00 KB)
 Trainable params: 11,521 (45.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 0.7637 - mae: 0.591
Epoch 2/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.4206 - mae: 0.427
Epoch 3/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 2.4298 - mae: 0.424
Epoch 4/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3691 - mae: 0.419
Epoch 5/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.3160 - mae: 0.385
Epoch 6/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3007 - mae: 0.380
Epoch 7/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3025 - mae: 0.379
Epoch 8/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2881 - mae: 0.370
Epoch 9/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2813 - mae: 0.364
Epoch 10/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2903 - mae: 0.373
Epoch 11/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2942 - mae: 0.366
Epoch 12/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2923 - mae: 0.369
Epoch 13/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2751 - mae: 0.360
Epoch 14/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2755 - mae: 0.359
Epoch 15/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2766 - mae: 0.359
Epoch 16/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2712 - mae: 0.356
Epoch 17/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3138 - mae: 0.372
```

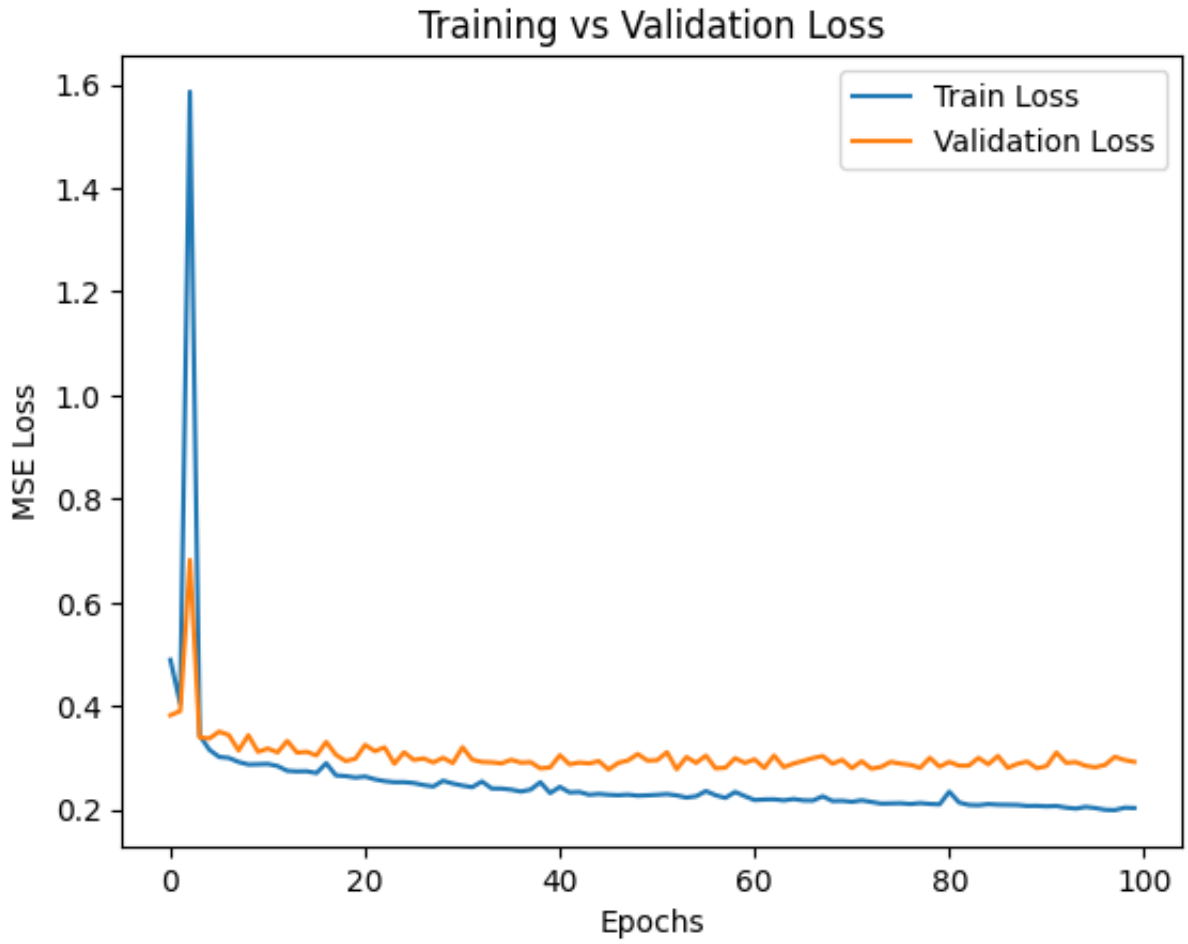
```
Epoch 18/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2631 - mae: 0.351
Epoch 19/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2576 - mae: 0.345
Epoch 20/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2601 - mae: 0.348
Epoch 21/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2635 - mae: 0.347
Epoch 22/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2514 - mae: 0.345
Epoch 23/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2584 - mae: 0.346
Epoch 24/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2549 - mae: 0.341
Epoch 25/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2493 - mae: 0.339
Epoch 26/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2481 - mae: 0.341
Epoch 27/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2479 - mae: 0.339
Epoch 28/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2464 - mae: 0.337
Epoch 29/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2562 - mae: 0.344
Epoch 30/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2478 - mae: 0.338
Epoch 31/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2470 - mae: 0.339
Epoch 32/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2350 - mae: 0.334
Epoch 33/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2706 - mae: 0.343
Epoch 34/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2403 - mae: 0.334
Epoch 35/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2380 - mae: 0.334
Epoch 36/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2395 - mae: 0.332
Epoch 37/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2366 - mae: 0.332
Epoch 38/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2384 - mae: 0.334
Epoch 39/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2487 - mae: 0.332
Epoch 40/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2294 - mae: 0.328
Epoch 41/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2326 - mae: 0.326
Epoch 42/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2360 - mae: 0.333
Epoch 43/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2253 - mae: 0.325
Epoch 44/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2281 - mae: 0.326
```

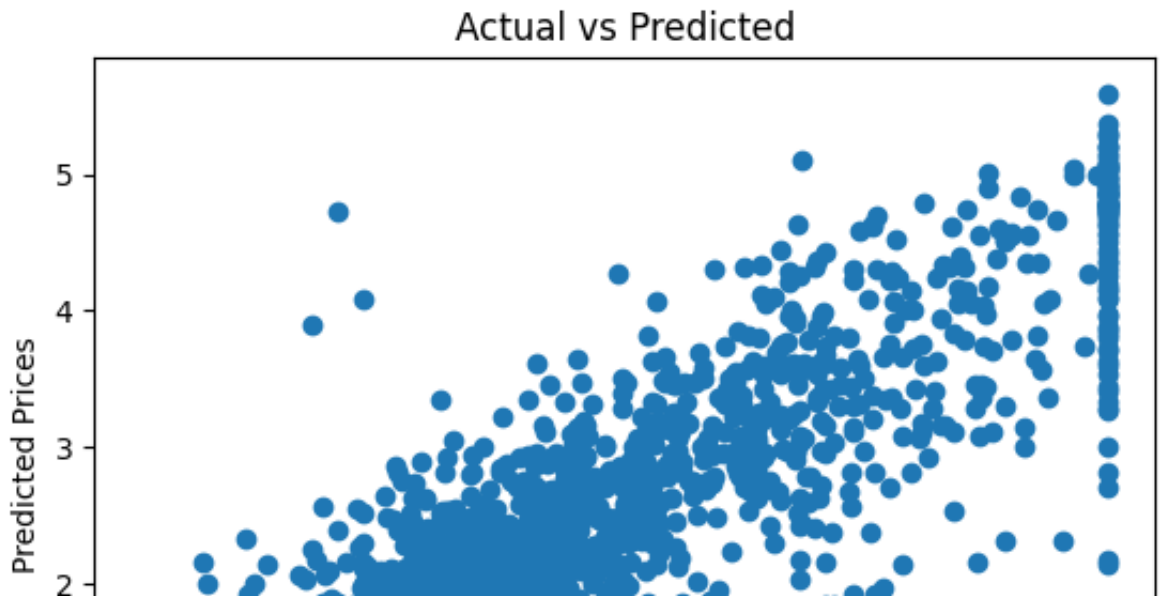
```
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2201 - mae: 0.320
Epoch 45/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2205 - mae: 0.324
Epoch 46/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2312 - mae: 0.330
Epoch 47/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.2342 - mae: 0.329
Epoch 48/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2239 - mae: 0.325
Epoch 49/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2225 - mae: 0.324
Epoch 50/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2392 - mae: 0.329
Epoch 51/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2346 - mae: 0.329
Epoch 52/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2316 - mae: 0.325
Epoch 53/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2302 - mae: 0.328
Epoch 54/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2229 - mae: 0.322
Epoch 55/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 0.2259 - mae: 0.326
Epoch 56/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2285 - mae: 0.326
Epoch 57/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2305 - mae: 0.329
Epoch 58/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2171 - mae: 0.320
Epoch 59/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2220 - mae: 0.321
Epoch 60/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2249 - mae: 0.323
Epoch 61/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2166 - mae: 0.318
Epoch 62/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2200 - mae: 0.320
Epoch 63/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2227 - mae: 0.321
Epoch 64/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2134 - mae: 0.318
Epoch 65/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2142 - mae: 0.319
Epoch 66/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2201 - mae: 0.321
Epoch 67/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2216 - mae: 0.324
Epoch 68/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2136 - mae: 0.318
Epoch 69/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2228 - mae: 0.317
Epoch 70/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2133 - mae: 0.317
Epoch 71/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2211 - mae: 0.321
```

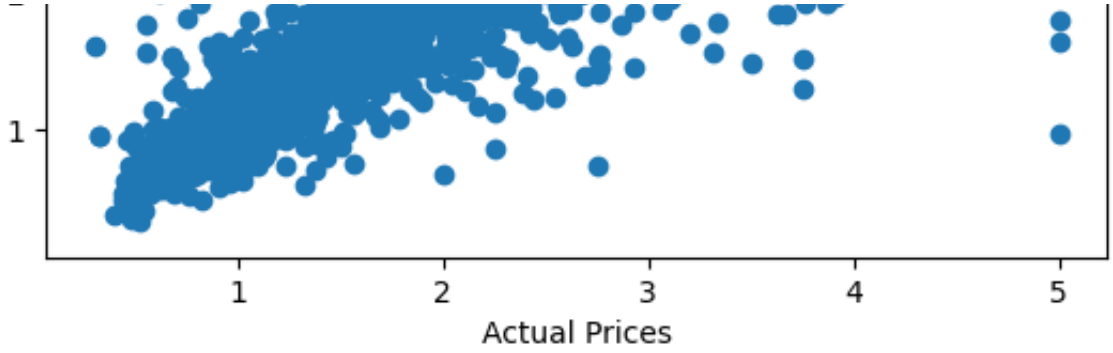
```
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2211 - mae: 0.321
Epoch 72/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2120 - mae: 0.317
Epoch 73/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2082 - mae: 0.313
Epoch 74/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2059 - mae: 0.312
Epoch 75/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2076 - mae: 0.315
Epoch 76/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2064 - mae: 0.314
Epoch 77/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2084 - mae: 0.313
Epoch 78/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2027 - mae: 0.310
Epoch 79/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2076 - mae: 0.312
Epoch 80/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2171 - mae: 0.319
Epoch 81/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2077 - mae: 0.310
Epoch 82/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2174 - mae: 0.320
Epoch 83/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2061 - mae: 0.311
Epoch 84/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2056 - mae: 0.313
Epoch 85/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2142 - mae: 0.319
Epoch 86/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2101 - mae: 0.316
Epoch 87/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2036 - mae: 0.313
Epoch 88/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2005 - mae: 0.308
Epoch 89/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1987 - mae: 0.309
Epoch 90/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2044 - mae: 0.312
Epoch 91/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2075 - mae: 0.312
Epoch 92/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1978 - mae: 0.305
Epoch 93/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2047 - mae: 0.308
Epoch 94/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2067 - mae: 0.311
Epoch 95/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1972 - mae: 0.306
Epoch 96/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2069 - mae: 0.310
Epoch 97/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2007 - mae: 0.309
Epoch 98/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2019 - mae: 0.309
```

```
523/523 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.2019 - mae: 0.309
Epoch 99/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1973 - mae: 0.308
Epoch 100/100
523/523 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2104 - mae: 0.315
65/65 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2552 - mae: 0.3435
65/65 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Test MAE: 0.3500092029571533
RMSE: 0.5242571142324666
R2 Score: 0.793908523254359
```







Printing after Normalization 80/20 Split

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]
```

```
# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert back to DataFrame for readable printing
X_train_scaled_df = pd.DataFrame(X_train, columns=df.drop("Price",
X_test_scaled_df = pd.DataFrame(X_test, columns=df.drop("Price", ax

print("\nAfter Normalization (First 5 rows of Training Data):")
print(X_train_scaled_df.head())

print("\nAfter Normalization (First 5 rows of Test Data):")
print(X_test_scaled_df.head())

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tr
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.010),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
```

```
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lati
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     3
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     3
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     3
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     3
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     3
```

```
   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422

After Normalization (First 5 rows of Training Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -0.326196  0.348490 -0.174916  -0.208365    0.768276  0.051376 -1.
1 -0.035843  1.618118 -0.402835  -0.128530   -0.098901 -0.117362 -0.
2  0.144701 -1.952710  0.088216  -0.257538   -0.449818 -0.032280 -0.
3 -1.017864  0.586545 -0.600015  -0.145156   -0.007434  0.077507 -1.
4 -0.171488  1.142008  0.349007   0.086624   -0.485877 -0.068832  0.

   Longitude
0   1.272587
1   0.709162
2  -0.447603
3   1.232698
4  -0.108551

After Normalization (First 5 rows of Test Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -1.155085 -0.286324 -0.520686  -0.171746   -0.030301  0.067408  0.
1 -0.708659  0.110435 -0.165815   0.223472    0.121851 -0.036030 -0.
2 -0.210402  1.856173 -0.610765   0.205892   -0.102419 -0.149989  1.
3  0.975113 -0.921138  0.305115  -0.176552    0.244979  0.030011 -0.
4 -0.081794  0.427842  0.024193  -0.158465   -0.319653 -0.052971  0.

   Longitude
0   0.285347
1   0.060975
2  -1.424870
3   0.429943
4  -1.170581
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential_5"

Layer (type)	Output Shape	P
dense_20 (Dense)	(None, 128)	
dense_21 (Dense)	(None, 64)	
dense_22 (Dense)	(None, 32)	
dense_23 (Dense)	(None, 1)	

Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.6666 - mae: 0.564
Epoch 2/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3951 - mae: 0.442
Epoch 3/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4124 - mae: 0.430
Epoch 4/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.3364 - mae: 0.405
Epoch 5/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3244 - mae: 0.399
Epoch 6/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3911 - mae: 0.414
Epoch 7/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3287 - mae: 0.397
Epoch 8/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3094 - mae: 0.383
Epoch 9/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3125 - mae: 0.385
Epoch 10/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2964 - mae: 0.372
Epoch 11/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2869 - mae: 0.367
Epoch 12/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2859 - mae: 0.366
Epoch 13/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.3104 - mae: 0.382
Epoch 14/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2886 - mae: 0.368
Epoch 15/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2966 - mae: 0.370
Epoch 16/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2833 - mae: 0.364
Epoch 17/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2805 - mae: 0.358
Epoch 18/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2720 - mae: 0.356
Epoch 19/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2653 - mae: 0.354
Epoch 20/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2702 - mae: 0.355
Epoch 21/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2667 - mae: 0.353
Epoch 22/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2711 - mae: 0.355
Epoch 23/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2564 - mae: 0.347
Epoch 24/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2587 - mae: 0.348
Epoch 25/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2616 - mae: 0.352
Epoch 26/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2545 - mae: 0.344
Epoch 27/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2661 - mae: 0.350
```

```
Epoch 28/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2507 - mae: 0.346
Epoch 29/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2606 - mae: 0.348
Epoch 30/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2458 - mae: 0.339
Epoch 31/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2578 - mae: 0.346
Epoch 32/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2550 - mae: 0.340
Epoch 33/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2554 - mae: 0.347
Epoch 34/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2581 - mae: 0.349
Epoch 35/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2466 - mae: 0.338
Epoch 36/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2397 - mae: 0.337
Epoch 37/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2488 - mae: 0.341
Epoch 38/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2387 - mae: 0.336
Epoch 39/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2524 - mae: 0.342
Epoch 40/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2422 - mae: 0.338
Epoch 41/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2418 - mae: 0.337
Epoch 42/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2419 - mae: 0.341
Epoch 43/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2496 - mae: 0.342
Epoch 44/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2441 - mae: 0.340
Epoch 45/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2496 - mae: 0.345
Epoch 46/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2379 - mae: 0.334
Epoch 47/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2374 - mae: 0.332
Epoch 48/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2572 - mae: 0.346
Epoch 49/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2423 - mae: 0.335
Epoch 50/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2446 - mae: 0.338
Epoch 51/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2415 - mae: 0.337
Epoch 52/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2273 - mae: 0.329
Epoch 53/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2310 - mae: 0.333
Epoch 54/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2295 - mae: 0.336
```

```
Epoch 55/100
 45/465 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2877 - mae: 0.363
```

Printing after Normalization 70/30 Split

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
```

```
# ==========================================
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Convert back to DataFrame for readable printing
X_train_scaled_df = pd.DataFrame(X_train, columns=df.drop("Price", a
X_test_scaled_df = pd.DataFrame(X_test, columns=df.drop("Price", axi

print("\nAfter Normalization (First 5 rows of Training Data):")
print(X_train_scaled_df.head())

print("\nAfter Normalization (First 5 rows of Test Data):")
print(X_test_scaled_df.head())

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_tra
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.015),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
```

```
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Lati
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     3
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     3
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     3
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     3
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     3

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422

After Normalization (First 5 rows of Training Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
```

```
0  0.133506  0.509357  0.181060  -0.273850   -0.184117 -0.010825 -0.
1 -0.532218 -0.679873 -0.422630  -0.047868   -0.376191 -0.089316 -1.
2  0.170990 -0.362745  0.073128  -0.242600   -0.611240 -0.044800 -0.
3 -0.402916 -1.155565  0.175848  -0.008560   -0.987495 -0.075230  1.
4 -0.299285  1.857152 -0.259598  -0.070993    0.086015 -0.066357  0.

   Longitude
0   0.780934
1   1.245270
2  -0.277552
3  -0.706938
4  -1.430902

After Normalization (First 5 rows of Test Data):
     MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  La
0 -1.152689 -0.283463 -0.507818  -0.169278   -0.031510  0.061278  0.
1 -0.706406  0.112947 -0.162520   0.213327    0.120220 -0.035516 -0.
2 -0.208307  1.857152 -0.595467   0.196308   -0.103428 -0.142154  1.
3  0.976831 -0.917719  0.295707  -0.173931    0.243007  0.026283 -0.
4 -0.079740  0.430075  0.022362  -0.156421   -0.320060 -0.051368  0.

   Longitude
0   0.286641
1   0.061963
2  -1.425909
3   0.431434
4  -1.171273
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

Model: "sequential_2"

Layer (type)	Output Shape	P
dense_8 (Dense)	(None, 128)	
dense_9 (Dense)	(None, 64)	
dense_10 (Dense)	(None, 32)	
dense_11 (Dense)	(None, 1)	

```
 Total params: 11,521 (45.00 KB)
 Trainable params: 11,521 (45.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 1.6104 - mae: 0.840
Epoch 2/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.4024 - mae: 0.445
Epoch 3/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3531 - mae: 0.420
Epoch 4/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3415 - mae: 0.410
Epoch 5/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3207 - mae: 0.396
Epoch 6/100
```

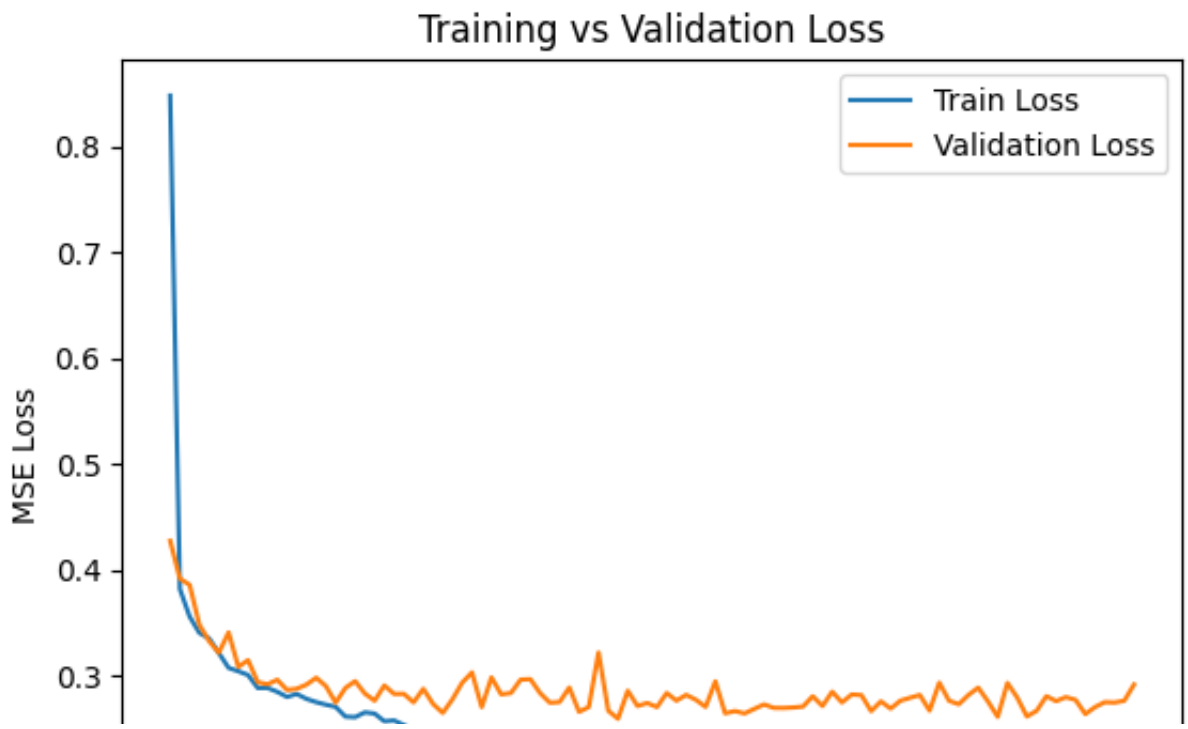
```
Epoch 6/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3282 - mae: 0.393
Epoch 7/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3169 - mae: 0.387
Epoch 8/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3052 - mae: 0.384
Epoch 9/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2989 - mae: 0.374
Epoch 10/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2853 - mae: 0.366
Epoch 11/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2875 - mae: 0.368
Epoch 12/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2950 - mae: 0.371
Epoch 13/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2936 - mae: 0.370
Epoch 14/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2826 - mae: 0.363
Epoch 15/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2757 - mae: 0.358
Epoch 16/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2685 - mae: 0.353
Epoch 17/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2659 - mae: 0.352
Epoch 18/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2707 - mae: 0.353
Epoch 19/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2674 - mae: 0.355
Epoch 20/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2599 - mae: 0.350
Epoch 21/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2628 - mae: 0.355
Epoch 22/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2625 - mae: 0.350
Epoch 23/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2717 - mae: 0.352
Epoch 24/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2540 - mae: 0.342
Epoch 25/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2446 - mae: 0.338
Epoch 26/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2543 - mae: 0.342
Epoch 27/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2347 - mae: 0.332
Epoch 28/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2609 - mae: 0.346
Epoch 29/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.2399 - mae: 0.332
Epoch 30/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2265 - mae: 0.328
Epoch 31/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2395 - mae: 0.333
Epoch 32/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2389 - mae: 0.333
```

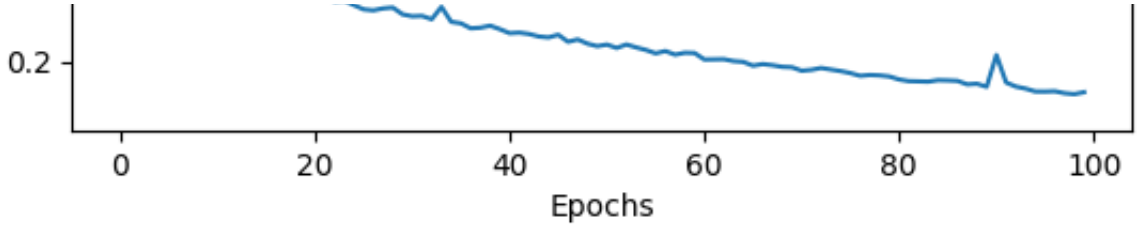
```
Epoch 33/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2420 - mae: 0.336
Epoch 34/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2374 - mae: 0.331
Epoch 35/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2388 - mae: 0.335
Epoch 36/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2340 - mae: 0.328
Epoch 37/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2292 - mae: 0.327
Epoch 38/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2256 - mae: 0.325
Epoch 39/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2377 - mae: 0.334
Epoch 40/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2348 - mae: 0.329
Epoch 41/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2261 - mae: 0.325
Epoch 42/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2207 - mae: 0.320
Epoch 43/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2117 - mae: 0.314
Epoch 44/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2124 - mae: 0.318
Epoch 45/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2199 - mae: 0.321
Epoch 46/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2262 - mae: 0.328
Epoch 47/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.2224 - mae: 0.323
Epoch 48/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2151 - mae: 0.315
Epoch 49/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2174 - mae: 0.319
Epoch 50/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2111 - mae: 0.311
Epoch 51/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2034 - mae: 0.310
Epoch 52/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2142 - mae: 0.315
Epoch 53/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2117 - mae: 0.312
Epoch 54/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2166 - mae: 0.315
Epoch 55/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2007 - mae: 0.309
Epoch 56/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.2030 - mae: 0.308
Epoch 57/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2025 - mae: 0.309
Epoch 58/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1999 - mae: 0.306
Epoch 59/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2027 - mae: 0.305
```

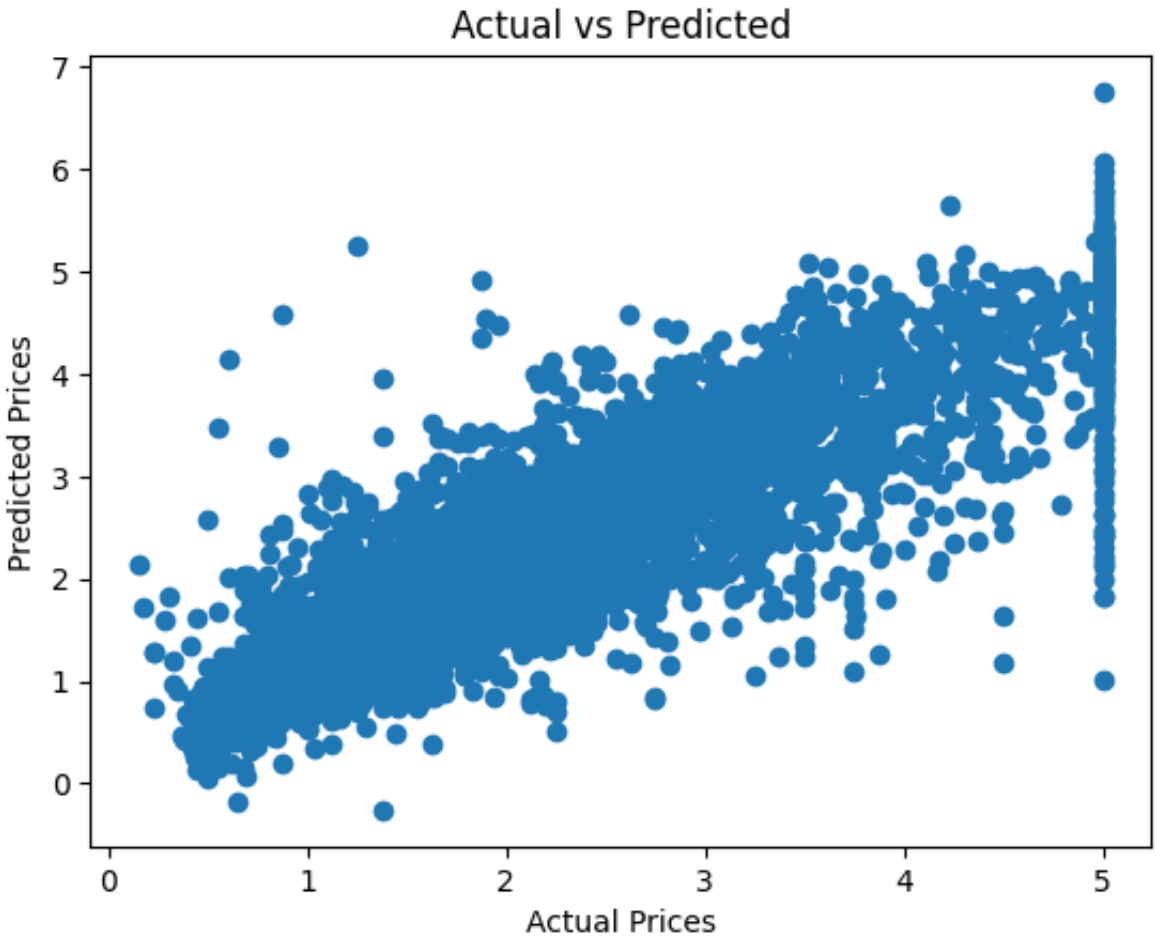
```
Epoch 60/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2079 - mae: 0.315
Epoch 61/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2008 - mae: 0.304
Epoch 62/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2008 - mae: 0.305
Epoch 63/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2023 - mae: 0.309
Epoch 64/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2061 - mae: 0.306
Epoch 65/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1926 - mae: 0.301
Epoch 66/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1921 - mae: 0.299
Epoch 67/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1869 - mae: 0.296
Epoch 68/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1983 - mae: 0.305
Epoch 69/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1901 - mae: 0.299
Epoch 70/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1962 - mae: 0.304
Epoch 71/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1773 - mae: 0.289
Epoch 72/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1826 - mae: 0.295
Epoch 73/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1948 - mae: 0.303
Epoch 74/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1853 - mae: 0.295
Epoch 75/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1852 - mae: 0.299
Epoch 76/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1868 - mae: 0.293
Epoch 77/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1836 - mae: 0.293
Epoch 78/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1775 - mae: 0.292
Epoch 79/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1900 - mae: 0.299
Epoch 80/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1868 - mae: 0.295
Epoch 81/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1826 - mae: 0.296
Epoch 82/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1781 - mae: 0.290
Epoch 83/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1746 - mae: 0.289
Epoch 84/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1716 - mae: 0.287
Epoch 85/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1849 - mae: 0.295
Epoch 86/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1691 - mae: 0.286
```

```
Epoch 87/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1783 - mae: 0.291
Epoch 88/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1741 - mae: 0.287
Epoch 89/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1765 - mae: 0.292
Epoch 90/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1744 - mae: 0.285
Epoch 91/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1710 - mae: 0.284
Epoch 92/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1956 - mae: 0.298
Epoch 93/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1876 - mae: 0.292
Epoch 94/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1737 - mae: 0.287
Epoch 95/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1702 - mae: 0.286
Epoch 96/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.1719 - mae: 0.286
Epoch 97/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1699 - mae: 0.283
Epoch 98/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1621 - mae: 0.279
Epoch 99/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1711 - mae: 0.286
Epoch 100/100
407/407 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1685 - mae: 0.283
194/194 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2891 - mae: 0.363
194/194 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Test MAE: 0.36201706528663635
RMSE: 0.5325324213337924
R2 Score: 0.7839374369717553
```




0.2
0
20
40
60
80
100
Epochs


Actual vs Predicted
Predicted Prices
Actual Prices
0
1
2
3
4
5
6
7